Retro Horse Race: API reference

One document for every interface of the game server. Machine-readable versions: `openapi.yaml` (the game server) and `openapi-wallet.yaml` (what a platform implements). Topic guides with more background: `WALLET_API.md`, `CLIENT_API.md`, `BACKOFFICE.md`, `ADMIN_CONSOLE.md`, `DEPLOY.md`.

Contents

1. Overview: who calls what, authentication, conventions
2. How a round works
3. For platforms: launch, wallet, webhooks, signing
4. Game API (the player's browser)
5. Client API (a platform's server)
6. Operator API (the game operator)
7. Back-office API (a client's staff)
8. Admin console API (the operator's staff)
9. Health and metrics
10. Error codes
11. Examples

---

1. Overview

1.1 Who calls what

Interface	Called by	Authentication	Section
Launch link `/?client=..&token=..`	the player's browser, sent by the platform	one-time launch token (validated by the platform's wallet)	3.1
Wallet API (`/session/validate`, `/wallet/*`)	**the game server calls the platform**	HMAC signature with the client's signing secret	3.2
Webhooks	**the game server calls the platform**	HMAC signature	3.3
Game API `/api/public`, `/api/auth`, `/api/me`, `/api/bet`, `/api/history`, `/api/round/:n`	the game page in the player's browser	session id (Bearer) from `/api/auth`; `public` and `round` need none	4
Client API `/api/client/*`	a platform's server	API key `hrk_<client>_<secret>` (Bearer)	5
Operator API `/api/admin`, `/api/command`, `/api/clients/*`, `/metrics`	the game operator's scripts	operator key (`X-Controller-Key` header or Bearer)	6
Back-office API `/bo/api/*`	the page at `/backoffice`	cookie session + CSRF token	7
Admin console API `/admin/api/*`	the page at `/admin`	cookie session + CSRF token	8

1.2 Conventions

- **Format**: JSON in, JSON out (`Content-Type: application/json`), UTF-8. Exceptions: CSV exports and `/metrics` (text).
- **Money**: whole numbers in **minor units** (100 = 1.00). Never floating point. The number of decimals of a currency is returned as `decimals` (2 for PHP/USD, 0 for JPY).
- **Time**: milliseconds since the Unix epoch (UTC), e.g. `1792583663575`. The server clock is the reference: `GET /api/public` returns `serverNow`.
- **Success / failure**: success has `"ok": true`; failure has `"ok": false, "error": "<CODE>"` (and sometimes `"message"`) with an HTTP status (see section 10). Branch on the code, not on the text.
- **Pair notation**: a winning combination is written low to high: `"1-2"` ... `"5-6"` (15 pairs). Any order of finish of the two horses counts for a bet; the winning pair finishes in the order it is written.
- **CORS**: `/api/*` answers `Access-Control-Allow-Origin: *` (headers `content-type, x-controller-key, authorization`). `/bo/api` and `/admin/api` are same-origin only. Browsers must not hold API keys: call the Client and Operator APIs from servers.
- **Rate limits**: a player session may send at most 12 bet requests per 5 seconds (`TOO_MANY_REQUESTS`, 429). Failed API-key attempts block an address for 5 minutes after 10 tries. The reverse proxy adds its own limits (`deploy/nginx.conf`).
- **Idempotency**: money calls from the game to a platform carry a unique `txId`; the platform must apply each `txId` once (section 3.2). Calls to the game are not idempotent: do not retry `POST /api/bet` blindly (read `/api/me` first).
- **Versioning**: no version prefix yet. Additive changes (new fields) can appear at any time: ignore unknown fields.

1.3 Identifiers

Id	Meaning
`clientId`	the platform in the game's client registry, e.g. `acme`
`playerId`	the platform's own id of the player (letters, digits, `_ . @ -`, up to 64). Unique per client: the same id at two clients is two players.
`roundId` / `raceNo`	the race number (`roundId` is the same number as text). Increases for ever.
`betId`	one bet request (several pairs bet together share it); a single bet on a pair is `<betId>:<pair>`, e.g. `9f2c41aa:1-2`
`txId`	one wallet transaction: `bet:<race>:<betId>`, `win:<race>:<playerId>`, `refund:...`, `rollback:...`
`session`	the game's session id of a player (64 hex characters), different from the launch token

---

2. How a round works

One shared table runs continuous rounds, the same race for every client:

```
betting (25 s) -> draw at bet close -> countdown (6 s) -> race (30 s) -> results (10 s) -> next round
```

phase (in round)	Meaning	Bets
`betting`	open until `betCloseAt`	accepted
`closed`	drawn, waiting for the start (`startAt`)	`BETTING_CLOSED`
`racing`	the race is running	`BETTING_CLOSED`
`results`	the result is shown, wins are paid	`BETTING_CLOSED`

- **Odds**: 15 fixed payout values (3, 4, 5, 8, 10, 20, 30, 60, 80, 100, 125, 175, 250, 500, 1000) are shuffled over the 15 pairs every race (`round.odds`). A winning bet pays `floor(stake x odds)` including the stake.
- **RTP**: the operator sets a long-run return between 88% and 99%; the odds do not change. It is the same for every client.
- **Fairness**: before betting closes `round.commit` = SHA-256 of the secret seed is published. After the race `GET /api/round/:n` reveals the `seed`: `sha256(seed) == commit`, and the odds layout and the result can be recomputed (`server/rng.js`, `server/odds.js`).
- **Clients animate** the race from `race.seed` and `race.startAt`; the finishing `order` is only published about 2 seconds before the start.

---

3. For platforms: launch, wallet, webhooks, signing

3.1 Launch (redirect)

1. The player chooses the game in your lobby. Your backend creates a **one-time launch token** (random, short lived, tied to the player).
2. Redirect the browser to `https://GAME_HOST/?client=<clientId>&token=<token>` (`client` may be omitted when the game serves a single platform). The game removes the token from the address bar.
3. The game calls your `POST /session/validate` with the token, creates its own session, and the player bets.

3.2 Wallet API (you implement it; the game calls it)

All requests are `POST`, JSON, **signed** (3.4). Replies are `{"ok": true, ...}`; refuse with HTTP 4xx (or `ok:false` and an `error` code). `INSUFFICIENT_FUNDS` (HTTP 402) when the balance is too low.

Endpoint	When	Request	Reply
`POST /session/validate`	at launch	`{ "token": "..." }`	`{ ok, playerId, currency, balance, returnUrl, brandId?, decimals?, limits?, realityCheckMinutes?, blocked? }` (token single use; invalid: 401 `ok:false`)
`POST /wallet/balance`	now and then	`{ "playerId": "p1" }`	`{ ok, balance }`
`POST /wallet/debit`	a bet	transaction fields below, `amount`	`{ ok, balance }` or 402 `INSUFFICIENT_FUNDS`
`POST /wallet/credit`	a win or a refund	transaction fields, `amount`	`{ ok, balance }`
`POST /wallet/rollback`	a debit of unknown outcome	transaction fields, `txId` of the debit, `rollbackTxId`	`{ ok, balance, status: rolled_back / not_found / already_rolled_back }`

Transaction fields (debit, credit, rollback): `playerId`, `amount` (minor units), `txId`, `gameId` (`horse-race`), `transactionType` (`bet`, `win`, `refund`, `rollback`), `roundId`, `betId` / `betIds`, `refTxId`, `currency`, `brandId`, `meta` (the same data).

Rules: apply each `txId` once and return the same reply when it repeats; a debit whose answer was lost is cancelled with `/wallet/rollback`, and you must then refuse a later debit with that `txId` (HTTP 409 `TX_ROLLED_BACK`); credits are retried until you accept them. Full details: `WALLET_API.md`, `openapi-wallet.yaml`.

Optional `/session/validate` fields: `brandId` (copied to every call), `decimals`, `limits` (`minBet`, `maxBet`, `maxRoundStake`, `dailyLossLimit`, minor units), `realityCheckMinutes`, `blocked: true` (refuses the launch with `PLAYER_BLOCKED`).

3.3 Webhooks (the game calls you)

Configure a URL per client (`webhook.url`). `POST` JSON, signed like the wallet calls. Reply any 2xx. Anything else is retried with back-off (also after a restart), so **de-duplicate on `id`**.

```
{ "id": "horse-race:acme:round:42", "type": "round.settled", "createdAt": 1792583663575, "gameId": "horse-race", "clientId": "acme", "data": { } }
```

`type`	When	`data`
`round.settled`	every race, after settlement	`roundId`, `raceNo`, `combo`, `order`, `odds`, `oddsOfWinner`, `rtp`, `commit`, `seed`, `bets`, `staked`, `paid` (your own players' totals), `ledgerHash`, `ledgerLines`
`big.win`	a player's payout in one race reaches the client's `bigWinMin`	`roundId`, `raceNo`, `playerId`, `brandId`, `currency`, `stake`, `payout`, `odds`, `betIds`

`ledgerHash` is the newest hash of the game's tamper-evident ledger; keep it to anchor the audit log outside the game server.

3.4 Signing (wallet calls, webhooks)

Every call from the game to a platform carries:

Header	Value
`X-Timestamp`	unix time in milliseconds
`X-Nonce`	random hex, different for every request
`X-Signature`	hex `HMAC-SHA256(secret, timestamp + "\n" + nonce + "\n" + METHOD + "\n" + path + "\n" + sha256hex(rawBody))`
`X-Key-Id`	the key id, when one is configured

`path` is the URL path your server receives (e.g. `/horse-api/wallet/debit`), `rawBody` the JSON text exactly as received, `METHOD` upper case. **Reject** when the signature differs, when the timestamp is more than 5 minutes off, or when the nonce was already used in that window. Allow only the game server's IP and use HTTPS.

Node.js:

```
const crypto = require('crypto');
const seen = new Map();
function verify(req, rawBody, secret) {
```

```
  const ts = req.headers['x-timestamp'], nonce = req.headers['x-nonce'], sig = req.headers['x-signature'] ||
'';
  if (!ts || !nonce || Math.abs(Date.now() - Number(ts)) > 300000 || seen.has(nonce)) return false;
  const bodyHash = crypto.createHash('sha256').update(rawBody).digest('hex');
  const want = crypto.createHmac('sha256', secret).update(`${ts}\n${nonce}\nPOST\n${new URL(req.url,
'http://x').pathname}\n${bodyHash}`).digest('hex');
  const ok = sig.length === want.length && crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(want));
  if (ok) { seen.set(nonce, Date.now()); for (const [n, t] of seen) if (Date.now() - t > 300000)
seen.delete(n); }
  return ok;
}
```

Python:

```
import hashlib, hmac, time
SEEN = {}
def verify(headers, method, path, raw_body: bytes, secret: str) -> bool:
    ts, nonce, sig = headers.get('x-timestamp'), headers.get('x-nonce'), headers.get('x-signature', '')
    if not ts or not nonce or abs(time.time() * 1000 - int(ts)) > 300000 or nonce in SEEN: return False
    msg = f"{ts}\n{nonce}\n{method.upper()}\n{path}\n{hashlib.sha256(raw_body).hexdigest()}".encode()
    ok = hmac.compare_digest(sig, hmac.new(secret.encode(), msg, hashlib.sha256).hexdigest())
    if ok: SEEN[nonce] = time.time()
    return ok
```

PHP:

```
function verify(array $h, string $method, string $path, string $raw, string $secret): bool {
  $ts = $h['x-timestamp'] ?? ''; $nonce = $h['x-nonce'] ?? ''; $sig = $h['x-signature'] ?? '';
  if (!$ts || !$nonce || abs(microtime(true) * 1000 - (float)$ts) > 300000) return false;      // also check
the nonce against a cache
  $msg = $ts . "\n" . $nonce . "\n" . strtoupper($method) . "\n" . $path . "\n" . hash('sha256', $raw);
  return hash_equals(hash_hmac('sha256', $msg, $secret), $sig);
}
```

---

4. Game API (the player's browser)

Base: `https://GAME_HOST`. Player calls use `Authorization: Bearer <session>` from `/api/auth`.

GET /api/public

No authentication. The live state of the table; poll it about once a second (or less). Fields:

Field	Meaning
`mode`	`live`
`serverNow`	server time (ms)
`nextRaceNo`	the race number of the next race
`timings`	`{ countdown, race, results, bet }` in ms
`rules`	`{ combos[15], oddsValues[15], shuffleOdds, minBet, maxBet, maxRoundStake, currency }` (server defaults; a player's own limits come from `/api/auth`)
`round`	current round or `null`: `{ raceNo, phase, openAt, betCloseAt, startAt, commit, odds{pair: odds}, rtp, cheapRtp, cheapPairs[3] }`
`race`	the race to animate: `{ raceNo, startAt, seed, order }` (`order` is `null` until about 2 s before `startAt`)
`history`	last 8 results: `[{ raceNo, order[6] }]`
`reveals`	last 5 revealed draws: `[{ raceNo, seed, commit, combo, order, odds, rtp }]`

In standby or while starting: HTTP 503 `STANDBY` / `STARTING`.

POST /api/auth

Exchanges the launch token for a game session.

Request: `{ "token": "<launch token>", "client": "acme" }` (`client` optional with a single client). Reply:

```
{ "ok": true, "session": "9c1e...64 hex", "playerId": "p1", "returnUrl": "https://lobby.acme.example/",
  "currency": "PHP", "decimals": 2, "balance": 250000, "brandId": "brandA",
  "limits": { "minBet": 100, "maxBet": 100000, "maxRoundStake": 500000, "dailyLossLimit": 0 },
  "responsible": { "realityCheckMinutes": 30, "startedAt": 1792583663575, "staked": 0, "paid": 0,
"dailyNetLoss": 0 },
  "rules": { } }
```

Errors: `INVALID_CLIENT`, `CLIENT_DISABLED`, `CLIENT_REQUIRED` (400/403), `INVALID_TOKEN`, `INVALID_PLAYER_ID` (401), `PLAYER_BLOCKED` (403), `SHUTTING_DOWN` (503). The session lasts up to 12 hours and survives restarts. `returnUrl` is dropped when its host is not on the client's allowed list.

GET /api/me

Bearer session. The player's balance, limits, bets of the current round and the last result.

```
{ "ok": true, "playerId": "p1", "currency": "PHP", "decimals": 2, "balance": 245000, "limits": { },
"brandId": null, "responsible": { },
  "returnUrl": null, "raceNo": 42, "bets": [ { "combo": "1-2", "stake": 5000, "odds": 60, "status": "open",
"payout": 0 } ],
  "last": { "raceNo": 41, "combo": "3-5", "bets": [ { "combo": "3-5", "stake": 1000, "odds": 125, "status":
"won", "payout": 125000 } ] } }
```

`status`: `open`, `won`, `lost`. Error: `SESSION_EXPIRED` (401).

POST /api/bet

Bearer session. Places bets for the current round; the stake is debited from the wallet at once.

Request: `{ "bets": [ { "combo": "1-2", "stake": 5000 }, { "combo": "3-4", "stake": 5000 } ] }` (1 to 15 pairs, `stake` whole minor units within the player's limits). Reply: `{ "ok": true, "balance": 235000,`

`"bets": [ { "id": "9f2c41aa:1-2", "raceNo": 42, "combo": "1-2", "stake": 5000, "odds": 60, "status": "open", "payout": 0 } ] }`

Errors: `SESSION_EXPIRED`, `BETTING_CLOSED` (409), `BAD_REQUEST`, `BAD_BET` (stake or pair invalid or outside min / max), `MAX_BET_EXCEEDED` (per pair), `MAX_ROUND_STAKE_EXCEEDED` (per race), `DAILY_LOSS_LIMIT` (400), `INSUFFICIENT_FUNDS` (402), `PLAYER_BLOCKED` (403), `TOO_MANY_REQUESTS` (429), `WALLET_REJECTED`, `WALLET_UNAVAILABLE` (502, the player is not charged: an unknown debit is rolled back), `SHUTTING_DOWN` (503). A request is all-or-nothing. Do not repeat a request that timed out without checking `/api/me` first.

GET /api/history?limit=50

Bearer session. The player's last rounds (at most 200):

```
{ "ok": true, "playerId": "p1", "clientId": "acme", "rounds": [ { "raceNo": 41, "roundId": "41", "combo": "3-5", "order": [3,5,1,2,6,4], "commit": "ab12...", "staked": 1000, "paid": 125000,
  "bets": [ { "raceNo": 41, "betId": "9f2c41aa:3-5", "combo": "3-5", "stake": 1000, "odds": 125, "status": "won", "payout": 125000, "placedAt": 1792583663575, "currency": "PHP", "brandId": null } ] } ] }
```

`status`: `open`, `won`, `lost`, `refunded` (a refunded bet counts its stake as returned in `paid`).

GET /api/round/:raceNo

No authentication. The draw proof of a **finished** race (the seed is secret until then; before that `NOT_FOUND` 404): `{ "ok": true, "hash": "sha256", "raceNo": 41, "roundId": "41", "commit": "...", "seed": "...", "odds": { "1-2": 60, ... }, "rtp": 0.9, "combo": "3-5", "order": [3,5,1,2,6,4] }`. Verify: `sha256(seed) == commit`; `oddsLayout(seed, raceNo, oddsValues)` equals `odds`; `draw(seed, raceNo, buildForRtp(odds, rtp).weights)` gives `combo` and `order`. The script does all three: `node scripts/verify-draw.js 41 https://game.example.com`. Races before the "lower number first" rule picked the order of the two winners at random: for those the script accepts the pair in either order.

---

5. Client API (a platform's server)

For the platform's own back-end. Every call needs `Authorization: Bearer hrk_<client>_<48 hex>` and, when the client has an allow list, a request from an allowed address (behind a proxy the game must run with `TRUST_PROXY=1`). A client sees only its own players. Wrong keys: `INVALID_API_KEY` (401).

Call	Reply
`GET /api/client/info`	`{ ok, client }`: the client's settings (secrets are masked as `***abcd`, key hashes are never shown): `id, name, status, currency, limits, currencyLimits, realityCheckMinutes, bigWinMin, allowedIps, returnUrlHosts, wallet{type,url,keyId,auth}, webhook{url,events}, apiKeys[{id,createdAt,expiresAt,lastUsedAt}]`
`GET /api/client/report`	`{ ok, clientId, bets, staked, paid, ggr, rtp, byCurrency{cur:{staked,paid}}, activeSessions, pendingCredits, pendingRollbacks, blockedPlayers, currentRound, rtpSetting }`
`GET /api/client/history?playerId=p1&limit=50`	same shape as `/api/history` for that player
`POST /api/client/players/block { "playerId": "p1" }`	`{ ok, blocked: true, closed }`: ends the player's sessions and refuses new launches (self-exclusion)
`POST /api/client/players/unblock { "playerId": "p1" }`	`{ ok, blocked: false }`
`POST /api/client/players/close-sessions { "playerId": "p1" }`	`{ ok, closed }`

`playerId` must match `[A-Za-z0-9_.@-]{1,64}` (else `BAD_REQUEST` 400).

---

6. Operator API (the game operator)

Header `X-Controller-Key: <CONTROLLER_KEY>` (or `Authorization: Bearer <CONTROLLER_KEY>`). Restrict these paths by IP in the proxy. In `live` mode results can never be set.

GET /api/admin (alias /api/report)

The live report: `{ ok, serverNow, rtpSetting, rtpOfCurrentRace, bestBetRtp, cheapPairsRtp, evenSpreadRtp, oddsValues, winsByOdds, oddsProb, limits, rounds, bets, staked, paid, actualRtp, houseWin, wins{pair:n}, pendingCredits, pendingRollbacks, pendingWebhooks, blockedPlayers, byCurrency, byClient, activeSessions, recent[], commit, currentRound, ops{ role, health, alerts[], recentAlerts[], snapshot, wallet, clients, tickAge, lease, uptime, diskFreeMb } }`. On a standby instance: `{ ok, role: "standby", lease, standby: true }`.

POST /api/command

Body `{ "cmd": "...", ... }`:

cmd	Parameters	Reply
`setRtp`	`rtp` (0.88 to 0.99, or 88 to 99)	`{ ok, rtp, appliesFromRace, currentRaceRtp }`; `RTP_OUT_OF_RANGE` (400)
`previewRtp`	`rtp`	`{ ok, rtp, uniformRtp, bestRtp, cheapRtp, evenSpreadRtp, rows[{ odds, probability, rtp, cheap }] }`
`closeSessions`	`playerId`, `clientId?`	`{ ok, closed }`
`blockPlayer` / `unblockPlayer`	`playerId`, `clientId?`	`{ ok, blocked, closed? }`

`clientId` defaults to the implicit `default` client. Any other command: HTTP 403.

GET /api/admin/history?client=acme&playerId=p1&limit=50

Same as the player's history, for any player.

Clients (registry)

Call	Purpose
`GET /api/clients`	all clients with `stats` (client report) and `users` count; `defaultClient` tells whether the implicit client exists
`POST /api/clients` `{ "client": { ... } }`	create. Reply `{ ok, client, apiKey, walletSecret? }`: **shown once**
`GET /api/clients/:id`	`{ ok, client, report }`
`POST /api/clients/:id` `{ ...fields to change }`	update `name, currency, limits, currencyLimits, realityCheckMinutes, bigWinMin, allowedIps, returnUrlHosts, wallet, webhook`
`POST /api/clients/:id/rotate-key` `{ "graceHours": 24 }`	new API key (shown once); older keys keep working for the grace time (0 = revoke now)
`POST /api/clients/:id/revoke-key` `{ "keyId": "k1" }`	revoke one key
`POST /api/clients/:id/status` `{ "status": "active" or "disabled" }`	suspend / enable (sessions and launches stop at once)
`POST /api/clients/:id/delete` `{ "confirm": "<id>" }`	remove from the registry
`GET /api/clients/:id/users`, `POST .../users` `{ username, role, password? }`, `POST .../users/:u/reset-password`, `.../status`, `.../delete`	back-office users of the client (temporary password shown once)

Client object (create / update):

```
{ "id": "acme", "name": "Acme Casino", "currency": "PHP",
  "wallet": { "url": "https://platform.acme.example/horse-api", "secret": "<16+ chars, generated when
omitted>", "keyId": "k1", "auth": "hmac", "timeoutMs": 8000 },
  "webhook": { "url": "https://platform.acme.example/horse-events", "secret": "<defaults to the wallet
secret>", "events": "round.settled,big.win" },
  "limits": { "minBet": 100, "maxBet": 100000, "maxRoundStake": 500000, "dailyLossLimit": 0, "decimals": 2 },
```

```
  "currencyLimits": { "JPY": { "minBet": 100, "maxBet": 50000, "decimals": 0 } },
  "realityCheckMinutes": 30, "bigWinMin": 1000000, "allowedIps": ["203.0.113.0/24"], "returnUrlHosts":
["lobby.acme.example"] }
```

`id`: 2 to 32 characters, lower-case letters, digits and `-`. The wallet URL must be `https` (plain `http` only for localhost). `wallet.type: "mock"` (play money) only outside production with `ALLOW_MOCK_WALLET=1`. Validation errors: HTTP 400 `BAD_CLIENT` with a `message`.

GET /metrics

Prometheus text format (also with `Authorization: Bearer <CONTROLLER_KEY>`): wallet calls and latency per client and operation, pending queues and their age, ledger health, rounds, staked and paid per currency and client, RTP setting / actual / z-score, sessions, tick age, role.

---

7. Back-office API (a client's staff)

Used by the page `/backoffice`. Same-origin only. **Login**: `POST /bo/api/login { client, username, password, otp? }` sets an `HttpOnly; SameSite=Strict` cookie `bo_session` (`Secure` over HTTPS) and returns `{ ok, user, client, csrf, role, idleMinutes }`. Send `X-CSRF-Token: <csrf>` on every non-GET request. Sessions end after 30 minutes idle or 12 hours. Five wrong passwords lock an account for 10 minutes (`TOO_MANY_ATTEMPTS`, 429). `OTP_REQUIRED` (401) means a code from the authenticator app is needed.

Roles: **viewer** < **support** < **admin**. A temporary password must be changed first (`MUST_CHANGE_PASSWORD`, 403).

Call	Role	Purpose
`POST /bo/api/logout`, `GET /bo/api/me`	any	session
`POST /bo/api/password { current, next }`	any	change password (12+ characters); other sessions end
`POST /bo/api/totp/setup` / `totp/enable { otp }` / `totp/disable { password }`	any	two-step sign-in
`GET /bo/api/client`	viewer	the client's settings (read only)
`GET /bo/api/report`	viewer	totals (as the client report) plus `rounds[]` (last 30)
`GET /bo/api/rounds?limit=`	viewer	`[{ raceNo, combo, odds, order, at, players, bets, staked, paid }]`
`GET /bo/api/players?query=&limit=`	viewer	`{ total, players[{ playerId, bets, staked, paid, lastBetAt, online, blocked }] }`
`GET /bo/api/players/:id`	viewer	`{ blocked, sessions[], dailyNetLoss, totals, rounds[] }`
`GET /bo/api/export/players.csv`, `export/rounds.csv`	viewer	CSV download
`POST /bo/api/players/:id/block` / `unblock` / `close-sessions { reason? }`	support	player controls
`GET /bo/api/users`, `POST /bo/api/users { username, role }`	admin	list / create (temporary password shown once)
`POST /bo/api/users/:u/role { role }` / `reset-password` / `status { status }` / `delete`	admin	manage users (not yourself; not the last administrator)
`POST /bo/api/apikey/rotate { graceHours }`	admin	new API key for the client (shown once)
`GET /bo/api/audit?limit=`	admin	audit log of this client

8. Admin console API (the operator's staff)

Used by the page `/admin`. Same login model as section 7 at `/admin/api/` (cookie `admin_session`, no client id, accounts in `data/admins.json`). Roles: **viewer** (read only) < **support** (also suspend / enable a client, handle a client's back-office users) < **admin** (everything).

Call	Role	Purpose
`POST /admin/api/login { username, password, otp? }`, `logout`, `GET me`, `POST password`, `totp/setup|enable|disable`	any	as section 7
`GET /admin/api/overview`	viewer	health, alerts, game totals, RTP preview rows, all clients with stats and wallet health
`GET /admin/api/rtp/preview?value=0.93`	viewer	what an RTP setting would give
`POST /admin/api/rtp { rtp }`	admin	set the RTP (88% to 99%, from the next race)
`POST /admin/api/clients { client }`	admin	create a client (API key and wallet secret shown once)
`GET /admin/api/clients/:id`	viewer	client, report, users, wallet health, recent races, recent activity
`POST /admin/api/clients/:id { ...settings }`	admin	edit settings, wallet, webhook (an empty secret keeps the current one)
`POST /admin/api/clients/:id/status { status }`	support	suspend / enable
`POST /admin/api/clients/:id/rotate-key { graceHours }`, `revoke-key { keyId }`	admin	API keys
`POST /admin/api/clients/:id/rotate-wallet-secret`, `rotate-webhook-secret`	admin	new signing secrets (shown once)
`POST /admin/api/clients/:id/delete { confirm }`	admin	remove the client
`GET /admin/api/clients/:id/users`, `POST .../users { username, role }` (admin), `POST .../users/:u/reset-password` and `status` (support), `.../delete` (admin)		the client's back-office users
`GET /admin/api/users`, `POST /admin/api/users`, `POST /admin/api/users/:u/role|reset-password|status|delete`, `GET /admin/api/audit`	admin	staff accounts and the audit log

Every change is written to the tamper-evident ledger and the audit log; secret values never are.

---

9. Health and metrics

Call	Auth	Reply
`GET /healthz`	none	`{ ok, mode, role, reasons[] }`. **200** only for the healthy active instance; **503** for a standby, a starting or shutting-down instance, a broken ledger or a stalled round loop. Use it for load balancers and uptime checks.
`GET /metrics`	operator key	Prometheus metrics (section 6)

---

10. Error codes

Code	HTTP	Where	Meaning
`INVALID_TOKEN`	401	auth	the launch token is invalid, used or expired
`INVALID_PLAYER_ID`	401	auth	the platform returned an id with characters outside `[A-Za-z0-9_.@-]` or longer than 64
`INVALID_CLIENT` / `CLIENT_REQUIRED`	400	auth	unknown client id / the link has none and there are several
`CLIENT_DISABLED`	403	auth	the client is suspended
`PLAYER_BLOCKED`	403	auth, bet	self-excluded or blocked
`SESSION_EXPIRED`	401	me, bet, history	no valid session: launch again from the lobby
`BETTING_CLOSED`	409	bet	betting is not open
`BAD_REQUEST`	400	many	malformed input
`BAD_BET`	400	bet	stake not a whole number, outside min / max, or unknown pair
`MAX_BET_EXCEEDED` / `MAX_ROUND_STAKE_EXCEEDED` / `DAILY_LOSS_LIMIT`	400	bet	a limit would be exceeded
`INSUFFICIENT_FUNDS`	402	bet	the wallet balance is too low
`TOO_MANY_REQUESTS`	429	bet	more than 12 bet requests in 5 seconds
`WALLET_REJECTED`	502	bet	the platform refused the debit
`WALLET_UNAVAILABLE`	502	bet	the wallet did not answer; the player was not charged
`SHUTTING_DOWN`	503	auth, bet	the server is restarting
`STANDBY` / `STARTING`	503	all	this instance is not the active one yet
`NOT_FOUND`	404	round, clients, bo	unknown race (or seed not yet revealed) / client / user
`INVALID_API_KEY`	401	client API	missing, wrong, expired or disallowed address
`BAD_CLIENT`	400	clients	invalid client settings (see `message`)
`CLIENT_EXISTS`	409	clients	the id is taken
`RTP_OUT_OF_RANGE`	400	operator	RTP outside 88% to 99%
`BAD_LOGIN`, `OTP_REQUIRED`, `TOO_MANY_ATTEMPTS`, `NOT_LOGGED_IN`, `MUST_CHANGE_PASSWORD`, `BAD_PASSWORD`, `WEAK_PASSWORD`, `BAD_OTP`	400 to 429	back office, console	sign-in and password problems
`BAD_CSRF` / `BAD_ORIGIN`	403	back office, console	the request token or origin is wrong

Code	HTTP	Where	Meaning
`FORBIDDEN`	403	back office, console	the role is too low
`SELF` / `LAST_ADMIN` / `USER_EXISTS` / `BAD_USER` / `CONFIRM`	400 / 409	users, clients	user-management guards
`SERVER_ERROR`	500	all	unexpected: see the server log

---

11. Examples

Launch a player (your backend): redirect to `https://game.example.com/?client=acme&token=3f9a...`

Place a bet from the browser

```
SESSION=$(curl -s https://game.example.com/api/auth -d '{"token":"3f9a...","client":"acme"}' | jq -r
.session)
curl -s https://game.example.com/api/bet -H "authorization: Bearer $SESSION" \
  -d '{"bets":[{"combo":"1-2","stake":5000},{"combo":"3-4","stake":5000}]}'
```

Read the table state

```
curl -s https://game.example.com/api/public | jq '{phase: .round.phase, race: .round.raceNo, closesIn:
((.round.betCloseAt - .serverNow)/1000)}'
```

Client report and a player's history (your server)

```
KEY=hrk_acme_5d2833910d569945a71e54f2a8958ba55b1b639a3369e57e
curl -s https://game.example.com/api/client/report -H "authorization: Bearer $KEY"
curl -s "https://game.example.com/api/client/history?playerId=p1&limit=20" -H "authorization: Bearer $KEY"
curl -s https://game.example.com/api/client/players/block -H "authorization: Bearer $KEY" -d
'{"playerId":"p1"}'
```

Verify a draw (anyone)

```
curl -s https://game.example.com/api/round/41 | jq '{commit, seed}'
echo -n "<seed>" | sha256sum        # must equal commit (hash the seed as text, the 64 hex characters)
node scripts/verify-draw.js 41 https://game.example.com      # checks the commit, the odds layout and the
draw
```

Operator: set the RTP, create a client

```
curl -s https://game.example.com/api/command -H 'x-controller-key: <key>' -d '{"cmd":"setRtp","rtp":0.93}'
curl -s https://game.example.com/api/clients -H 'x-controller-key: <key>' \
  -d '{"client":{"id":"acme","name":"Acme Casino","currency":"PHP","wallet":
{"url":"https://platform.acme.example/horse-api"}}}'
```

Wallet debit as the game sends it (what your endpoint receives)

```
POST /horse-api/wallet/debit HTTP/1.1
Content-Type: application/json
X-Timestamp: 1792583663575
X-Nonce: 6a1f0c...
X-Signature: 9b3e...
X-Key-Id: k1

{"playerId":"p1","amount":5000,"txId":"bet:42:9f2c41aa","gameId":"horse-
```

```
race","transactionType":"bet","roundId":"42","betId":"9f2c41aa","betIds":["9f2c41aa:1-
2"],"currency":"PHP","brandId":null,"meta":{"raceNo":42,"kind":"bet"}}
```